Doc bibliothèque Pic +

Idée principale de Pic+ :

- avoir accès à des fonctions basiques et facilitées
- rendre le code plus lisible
- rendre le développement plus rapide dans le cadre de la micro-informatique à l'IG2I

Informations de préambule :

- Si un mot est en *italique*, il a donc été défini comme mot-clé du langage.
- Si un mot est en **gras**, c'est que c'est du code ou une variable.

Sommaire :

1. Installation dans le projet

2. Modules

2.1. Commun

type **octet**

type **t_port**

fonction **perte_temps**

fonction **confINT0**

2.2. AnalogPort

fonction **confAnalogTomV**

fonction **analogTomV**

fonction **confAnalog**

fonction **analogRead**

2.3. TMR0

fonction **confTMR0**

fonction **confTMR0ms**

fonction **resetTMR0**

2.4. aff7seg

fonction **conf7seg**

fonction **convert7seg**

fonction **write7seg**

2.5. KeyPad

fonction **keyPadRead**

fonction **isKeySet**

fonction **keyBitmapChange**

fonction **keyBitmapToInt**

2.6. Sequential

fonction **initSequence**

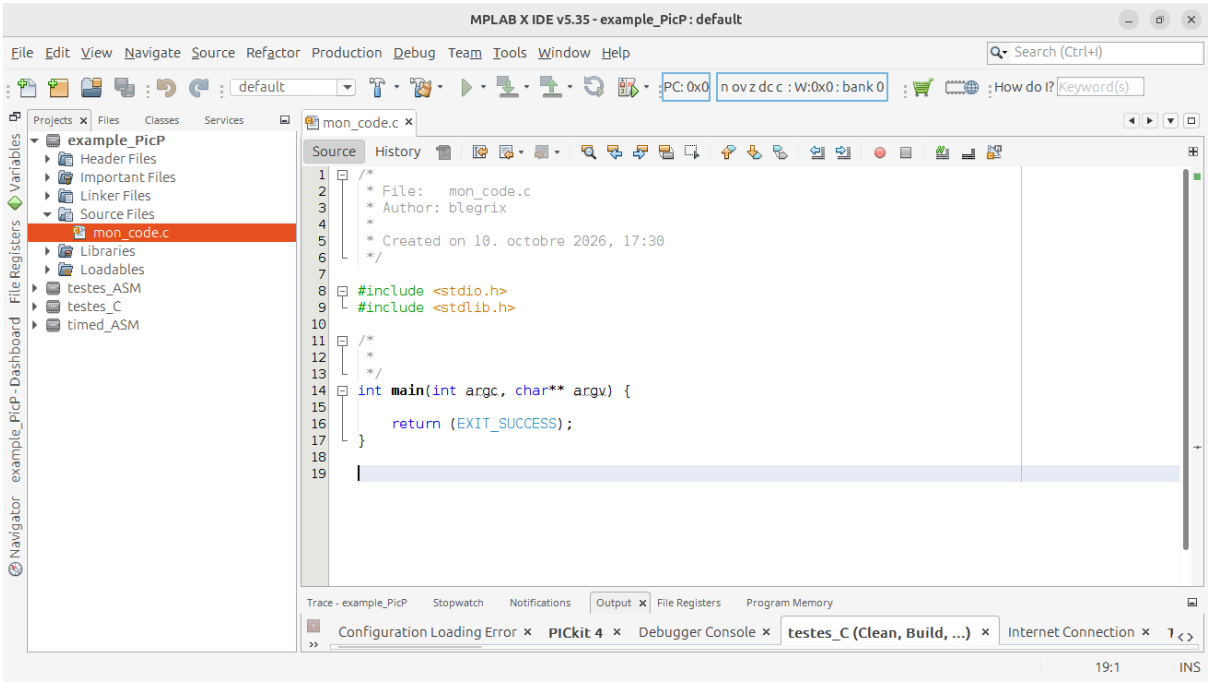
fonction **addSequenceStep**

fonction **SequenceStep**

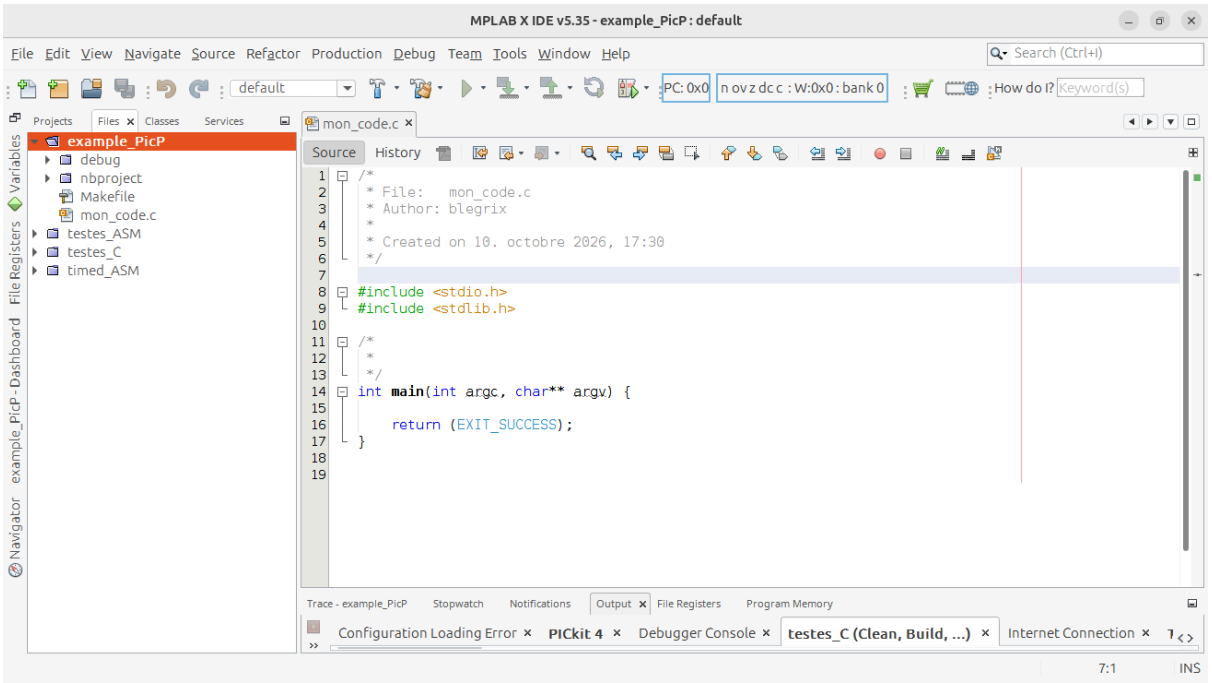
fonction **SequanceSetPos**

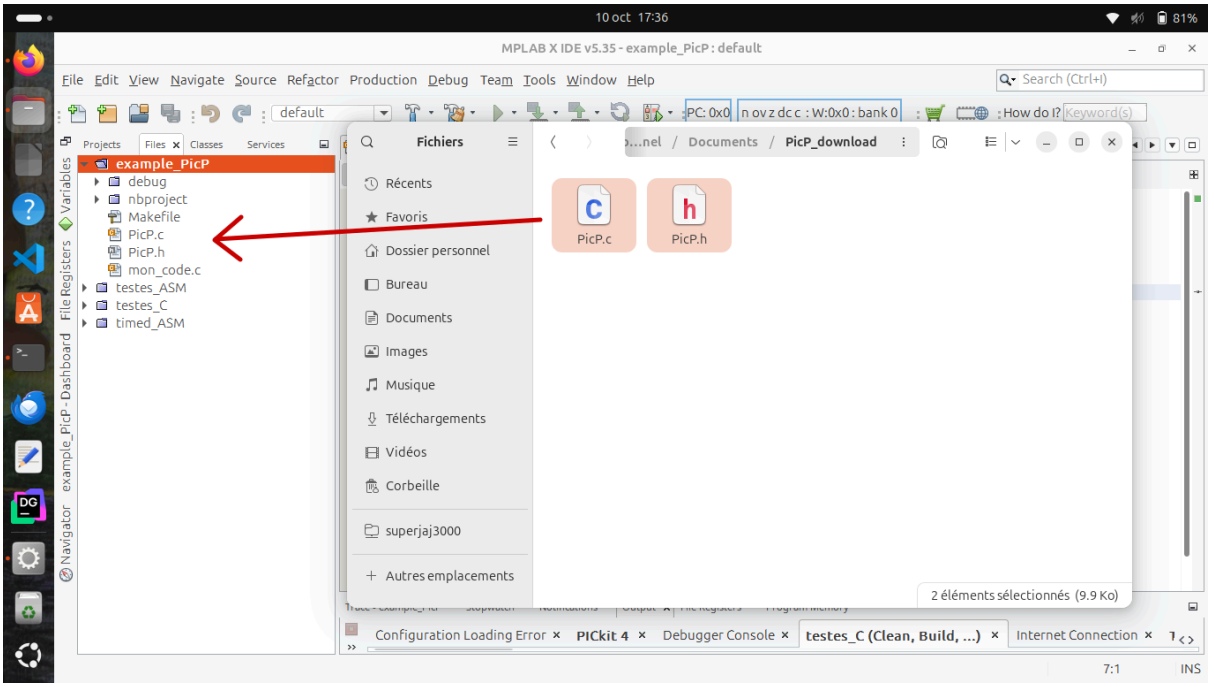
1) Installation dans le projet.

Nous allons montrer dans un projet imaginaire « example_PicP »

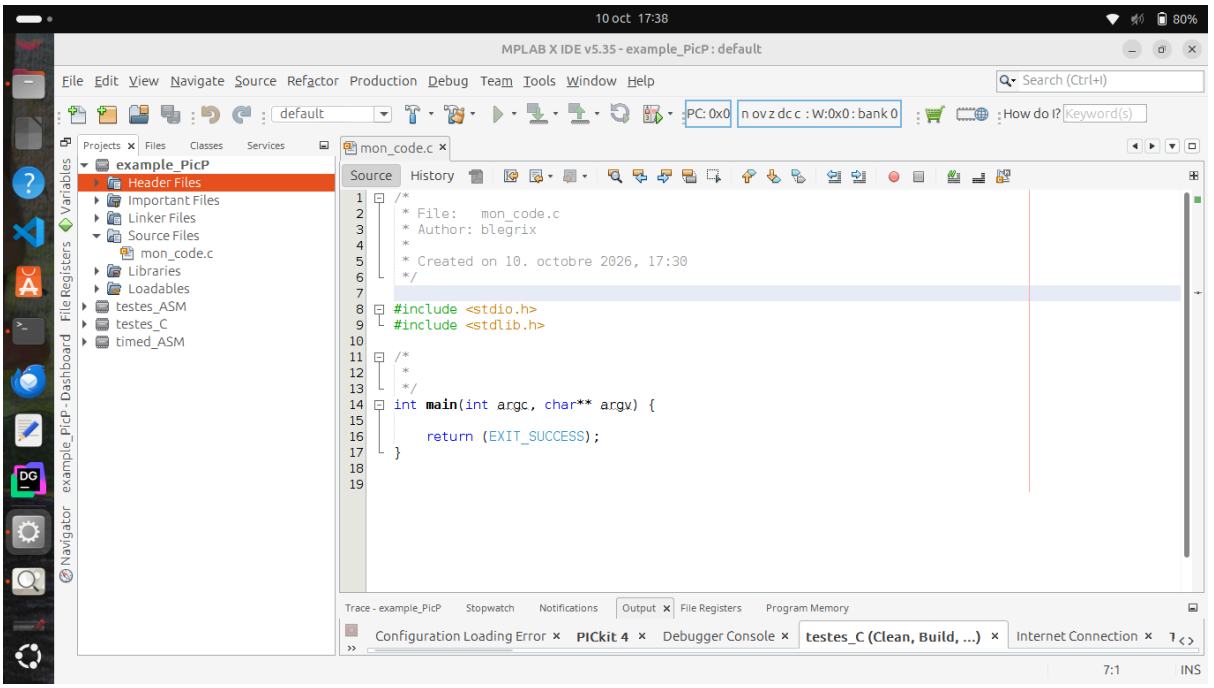


1. aller dans Folder et ajouter les fichiers PicP.h et PicP.c à la racine

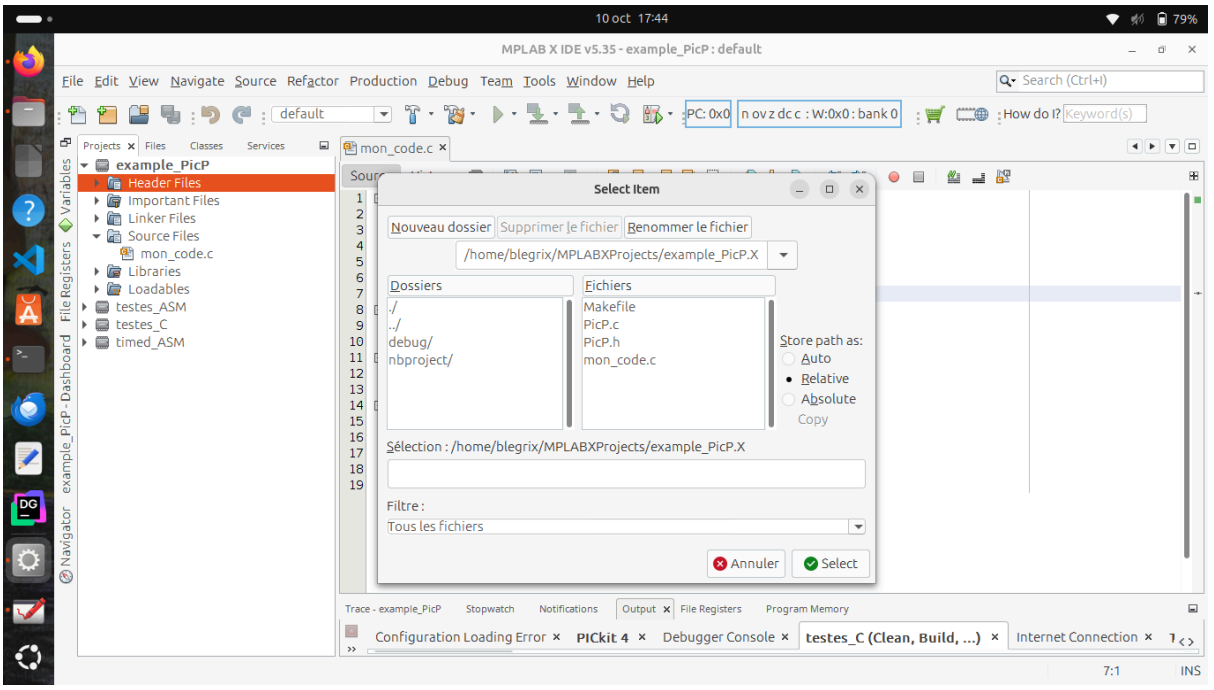




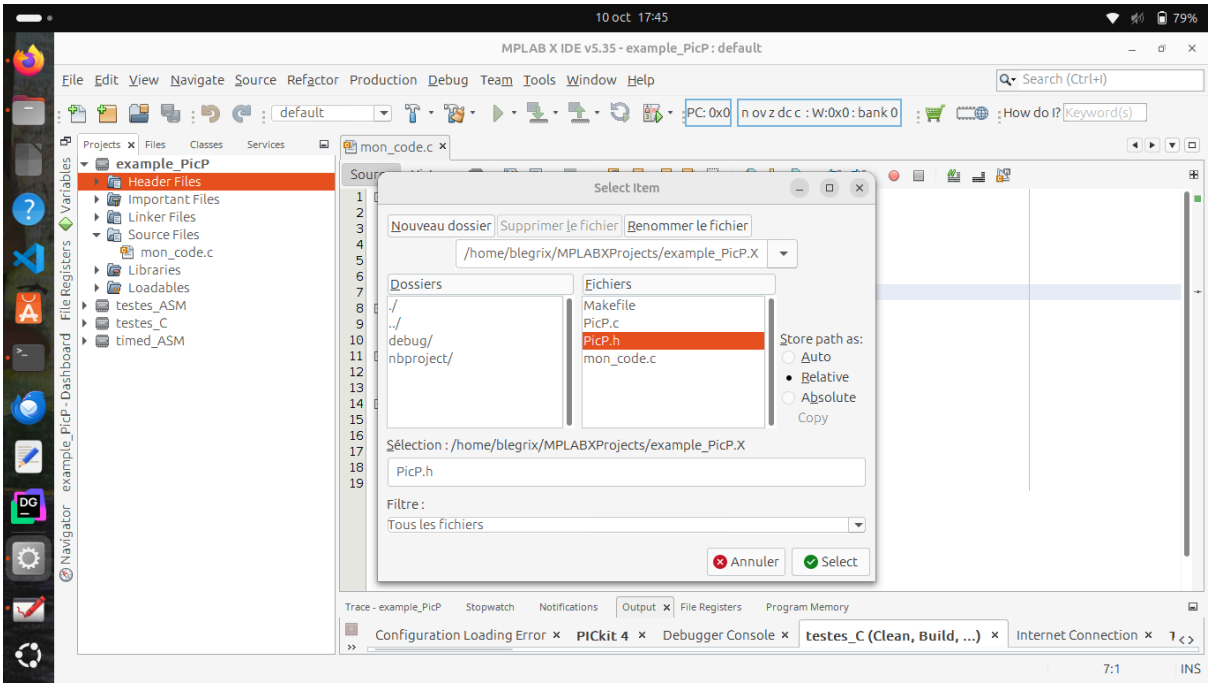
2. aller dans Projects



3. ajouter au projet PicP.h dans les Header Files
   3.1. clic droit sur « Header Files »
   3.2. clic sur « Add Existing Item... »

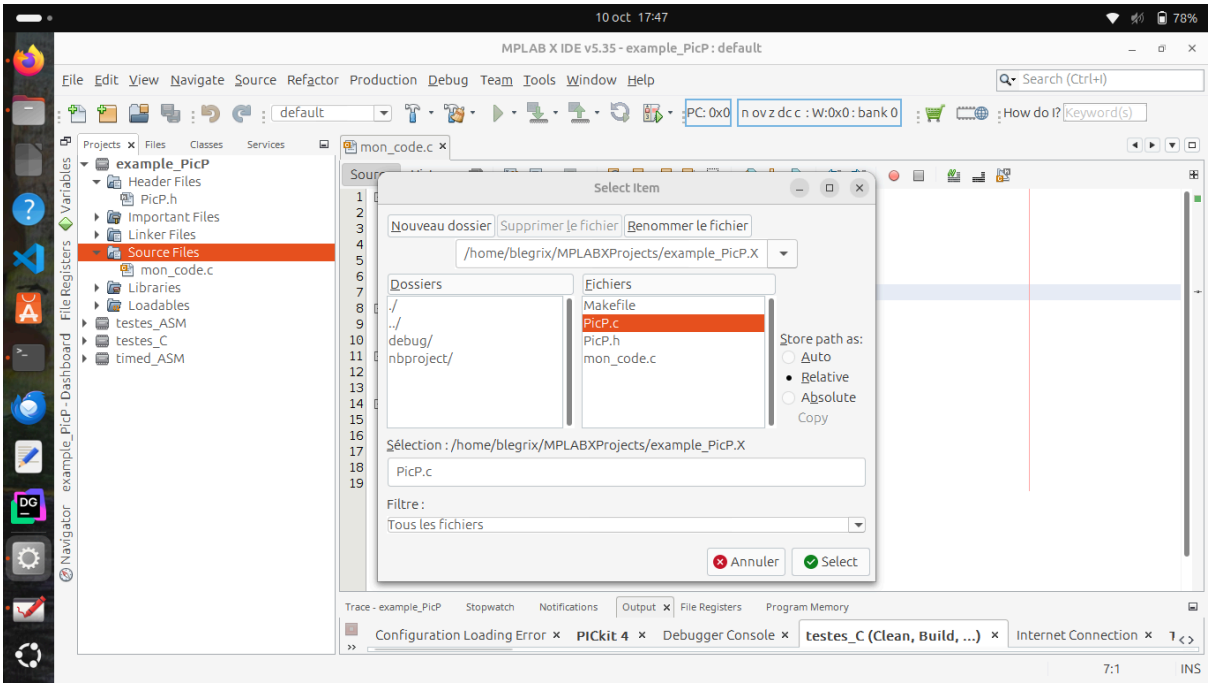


3.3. clic sur PicP.h

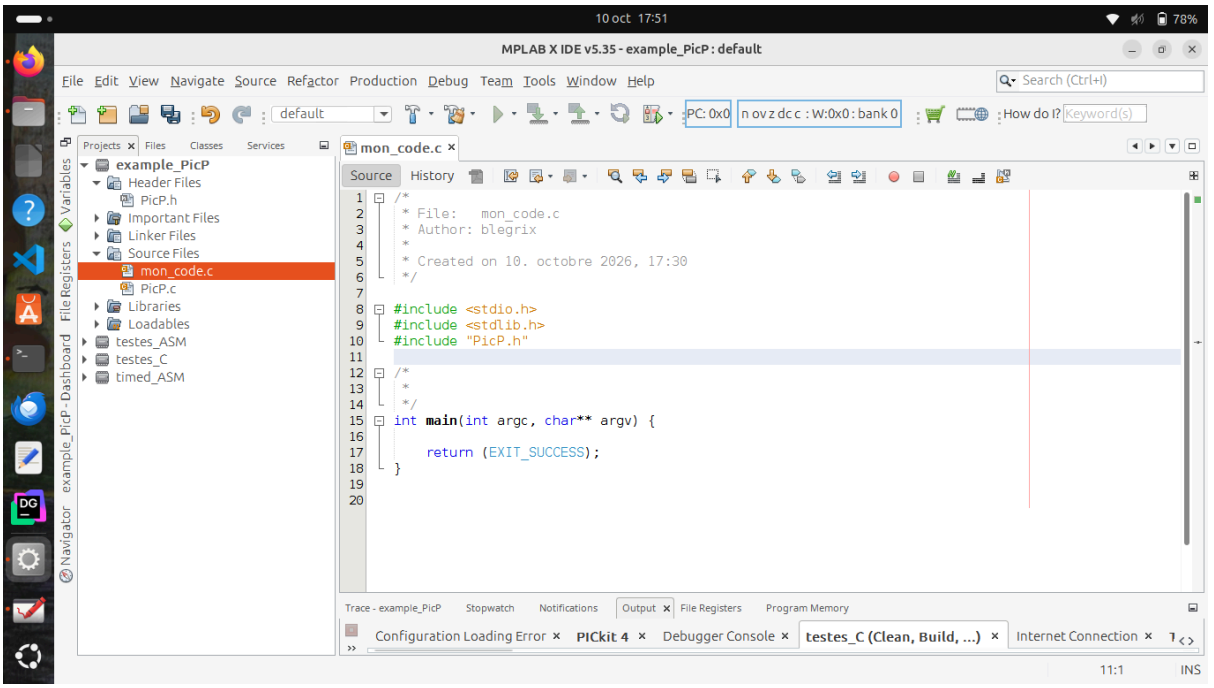


3.4. puis clic sur Select

4. on fait la même chose avec PicP.c dans Source Files



5. on n'a plus qu'à importer la lib dans notre code source.
   5.1. aller dans votre fichier votre_code.c
   5.2. avec les inclusions, ajouter la ligne
   #include "PicP.h"



6. la lib est installée, vous pouvez donc utiliser les modules ci-dessous en les activant en décommentant leur définition dans PicP.h ;

2) Modules

Chaque module hormis commun doit être activé dans PicP.h en décommentant le

#define [nom_du_module]_module

Commun :

Le module commun contient des fonctions utiles en permanence comme :

type **octet** c'est un unsigned char qui ne représente qu'un octet non signé.

Type **t_port** : c'est un volatile unsigned char qui représente les PORTX et TRISX pour les affecter.

void perte_temps(int tempo);
Effectue une perte de temps processeur proportionnelle à (tempo).

Exemple : vous avez besoin d'un délai entre deux actions :

prog{
action1();
tempo(200);
action2();
}

void confINT0(octee EN_TMR0,octee EN_INT0);
Active ou non les interruptions **TMR0** et **INT0**.

Variables ->

EN_TMR0 -> *enable* = l'interruption **TMR0** est configurée et allumée
disable = l'interruption **TMR0** est éteinte.

EN_INT0 -> *enable* = l'interruption **INT0** est configurée et allumée
disable = l'interruption **INT0** est éteinte.

Exemple : vous avez besoin de l'interruption physique 0 mais pas de l'interruption TMR0 :
init{
confINT0(disable,enable);
}

AnalogPort

Ce module permet de gérer les entrées analogiques (CAN/ADC) et la conversion en millivolts (mV).
Macros de sélection des canaux analogiques (Pins) :

• **AN0** à **AN7** (valeurs binationales de 0b000 à 0b111) désignent les canaux d'entrée analogique 0 à 7.

#define defaultConfAnalogTomV 21.8,5085.5
Configuration par défaut de la conversion tension/binaire (tension min = 21.8 mV, tension max = 5085.5 mV).

void confAnalogTomV(double min, double max);
Redéfinit la configuration de calcul de **analogTomV** en fonction des tensions minimale (min) et maximale (max) exprimées en mV.

int analogTomV(int analog);
Convertit la valeur binaire lue via **analogRead** en une valeur de tension équivalente en millivolts (mV) selon la configuration définie.

void confAnalog(octet mode_10bit, octet PCFG);
Configure les entrées analogiques du microcontrôleur. **mode_10bit** définit la résolution de lecture (10-bit ou 8-bit) et **PCFG** configure le registre de sélection des broches analogiques/numériques.

rappel de PCFG:

Définition rapide	PCFG<3:0>	AN7	AN6	AN5	AN4	AN3	AN2	AN1	AN0	VREF+	VREF-
PCFG_ALL	0b0000	A	A	A	A	A	A	A	A	VDD	VSS
	0b0001	A	A	A	A	VREF+	A	A	A	AN3	VSS
PCFG_A0_A4	0b0010	D	D	D	A	A	A	A	A	VDD	VSS
	0b0011	D	D	D	A	VREF+	A	A	A	AN3	VSS
PCFG_A0_1_3	0b0100	D	D	D	D	A	D	A	A	VDD	VSS
	0b0101	D	D	D	D	VREF+	D	A	A	AN3	VSS
PCFG_no_port	0b011x	D	D	D	D	D	D	D	D	—	—
	0b1000	A	A	A	A	VREF+	VREF-	A	A	AN3	AN2
PCFG_A0toA5	0b1001	D	D	A	A	A	A	A	A	VDD	VSS
	0b1010	D	D	A	A	VREF+	A	A	A	AN3	VSS
	0b1011	D	D	A	A	VREF+	VREF-	A	A	AN3	AN2
	0b1100	D	D	D	A	VREF+	VREF-	A	A	AN3	AN2
	0b1101	D	D	D	D	VREF+	VREF-	A	A	AN3	AN2
PCFG_A0	0b1110	D	D	D	D	D	D	D	A	VDD	VSS
	0b1111	D	D	D	D	VREF+	VREF-	D	A	AN3	AN2

int analogRead(octet Pin);
Effectue une conversion analogique-numérique et renvoie la valeur binaire lue sur le canal passé en paramètre (ex: **AN0**).

Exemple : lire la tension en mV sur le canal AN0 et effectuer une action si elle dépasse 2500 mV :

```
init{
        confAnalog(enable, 0b1110); // config des entrées analogiques
        confAnalogTomV(0, 5000);    // plage de 0V à 5V (0 à 5000 mV)
}
boucle{
        int valBinaire = analogRead(AN0);
        int tensionmV = analogTomV(valBinaire);
        if (tensionmV > 2500) {
                action();
        }
}
```

TMR0

Ce module traite du fonctionnement du timer 0.

void confTMR0(octet PS, unsigned short reset_delay);
Configure le TMR0 et l'active. (Ancienne version, fonctionnelle)
Variables ->

PS alias Pré Scaleur -> renseigne le binaire du prescaler
Voici un tableau des prescalers disponibles :

Nom de la macro	Valeur du prescaler
NPS_0	Pas de prescaler
PS2_0	1/2
PS4_0	1/4
PS8_0	1/8
PS16_0	1/16
PS32_0	1/32
PS64_0	1/64
PS128_0	1/128
PS256_0	1/256

Rappel des méthodes de choix :
Si on veut un timer de X ms :
PS = la valeur la plus proche au-dessus de 1000X / 65535

reset_delay = 65535 - 1000X * PS

void confTMR0ms(int ms);
Configure le TMR0 en donnant le nombre de ms à stocker.

Variables -> **ms** -> nombre de ms entre chaque interruption

void resetTMR0(void);
Permet de réinitialiser le TMR0 et l'interruption si elle est utilisée ; ne prend pas d'arguments.

Exemple : on veut effectuer une tâche toutes les secondes :

```
init{
        confTMR0ms(1000); // config toutes les secondes
        confINT0(enable,disable);
}
interupt{
        if(TMR0IF){
                resetTMR0();
                tache();
        }
}
```

aff7seg

Ce module s'occupe de l'affichage.

Câblage :
PORT1 <-> Led
PORT2[N:N+3] <-> dis[0:3]

Tris:
TRIS1 = 0x00;
TRIS2 = 0b…0000…
| |
N+3 N

void conf7seg(t_port *port, t_port *dis, octet pos_dis0);
Configure les ports du module.

Variables ->
port -> &PORT1
dis -> &PORT2
pos_dis0 -> N

octet convert7seg(octet chiffre);
Convertit **chiffre** et renvoie le code LED d'un 7 segments.

void write7seg(int val, octet points);
Rafraîchit les LED du 7 segments.

Variables :
- **val** -> valeur à afficher ;
- **points** -> bitmap de 4 bits pour placer les points

Exemple : afficher 10.24 sur un 7 segments :
Câblage :
- PORTD <-> Led
- PORTC[0:3] <-> dis[0:3]

```
init{
    TRISD = 0x00;
    TRISC = 0b0000;
    conf7seg(&PORTD,&PORTC,0);
}
boucle{
    write7seg(1024,0b0100);
}
```

KeyPad

Module permettant de lire un KeyPad.

Câblage :
- PORT1 <-> KeyPad

Tris:
- TRIS1 = 0x0f;

type **t_key_bitmap** est un bitmap des touches appuyées.

Touche	Macro	Binaire	Valeur Int
0	*KEY_0*	0b0000000000000010	0
1	*KEY_1*	0b0001000000000000	1
2	*KEY_2*	0b0010000000000000	2
3	*KEY_3*	0b0100000000000000	3
4	*KEY_4*	0b0000000100000000	4
5	*KEY_5*	0b0000001000000000	5
6	*KEY_6*	0b0000010000000000	6
7	*KEY_7*	0b0000000000010000	7
8	*KEY_8*	0b0000000000100000	8
9	*KEY_9*	0b0000000001000000	9
A	*KEY_A*	0b1000000000000000	10
B	*KEY_B*	0b0000100000000000	11
C	*KEY_C*	0b0000000010000000	12
D	*KEY_D*	0b0000000000001000	13
*	*KEY_Star*	0b0000000000000001	14
#	*KEY_Hashtag*	0b0000000000000100	15
Toutes	*all_KEY*	0b1111111111111111	*Default_KBTI*, cas null/plusieurs touches

Avec l'opérateur ~, on peut obtenir la négation. Ex : *~KEY_Star* => toute touche sauf Star
Default_KBTI est définie par défaut à 16.

t_key_bitmap keyPadRead(t_port *port);
Effectue la lecture d'un keypad et l'écrit au format bitmap.

Variables :
port = &PORT1
return : le bitmap de l'ensemble des touches appuyées.

octet isKeySet(t_key_bitmap scan, t_key_bitmap key);
Renvoie si la touche **key** est active dans le **scan**.

Variables :
scan -> scan du read (le bouton appuyé) ou du change (le bouton a changé d'état)
key -> bitmap des touches à vérifier. Pour tester keyX ou keyY, vous pouvez mettre :
KEY_X | *KEY_Y* et il retournera si keyX ou keyY est actif.
Return :
un booléen indiquant si l'une des touches est appuyée ou non.

t_key_bitmap keyBitmapChange(t_key_bitmap scan);
Génère un *t_key_bitmap* pour les changements de touches depuis le dernier appel de la fonction.
Variables :

scan -> le scan le plus récent du keypad.

octet keyBitmapToInt(t_key_bitmap val);
Analyse val et renvoie un octet en fonction de la valeur de la touche. Si plusieurs touches sont pressées, il reste sur le cas par défaut défini par Default_KBTI dans PicP.h dans le module.

Sequential

Module permettant de récupérer une série de **t_Step** dans l'ordre et donc de réaliser des systèmes séquentiels.

Type **t_Step** : type modifiable pour les types de données récupérées et rendues par Sequential.
Par défaut, c'est **octet**.

Le #define *maxSeqStep* est le nombre maximum d'instructions dans une séquence.
Par défaut, c'est 16 instructions.

Type **t_sequence** : type de données qui permet de sauvegarder et de traiter une séquence.

void initSequence(t_sequence *seq);
Initialise le **t_sequence** de la séquence que l'on veut créer.
Variable :

seq -> le pointeur vers notre séquence

void addSequenceStep(t_sequence *seq, t_Step Step);
Ajoute l'étape **Step** à **seq** dans la limite de *maxSeqStep*.

Variables :

seq -> pointeur vers la séquence à modifier
Step -> t_Step lié à l'étape

t_Step SequenceStep(t_sequence *seq);
Lit une instruction actuelle et passe la séquence **seq** a l'instruction suivante.

void SequanceSetPos(t_sequance* seq, octet pos);
permet de définir la position **pos** dans la séquence **seq**.